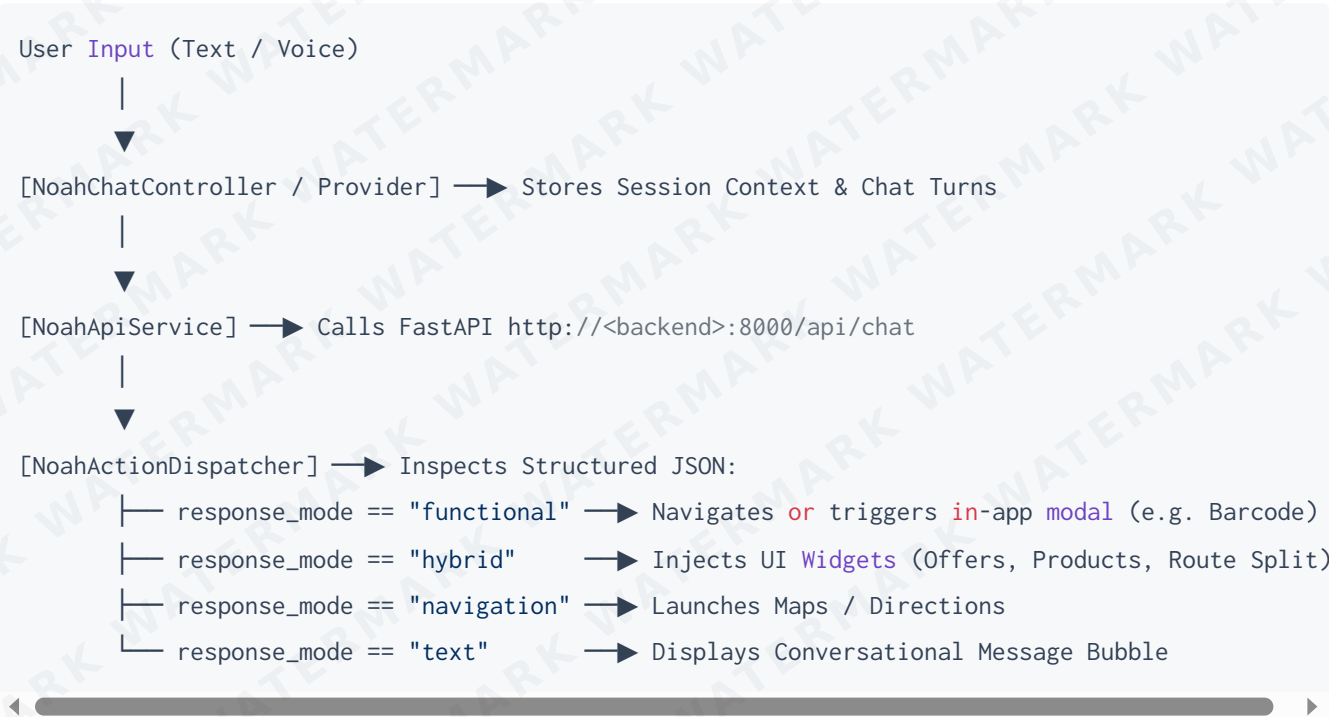


Noah AI Assistant — PayTo Flutter Integration Plan

1. Overview & Architecture

Noah functions as the embedded conversational intelligence inside the PayTo Flutter app. Noah interprets natural language requests, invokes PayTo application workflows, retrieves grounded facts from documents and the backend, and displays interactive UI elements (barcodes, offer carousels, store split breakdowns, maps).

Interaction Flow



2. Target File Structure in Flutter

```
lib/
├── models/
│   └── noah_chat_models.dart # Pydantic-equivalent Dart data classes
├── services/
│   ├── noah_api_service.dart # Backend API client with offline retry
│   └── noah_action_dispatcher.dart # Maps planner_actions to Flutter screens/actions
├── providers/
```

```

|   └─ noah_chat_provider.dart      # Multi-turn state, history & active context
└─ screens/
|   └─ noah_chat_screen.dart        # Main chat UI / expandable bottom sheet
└─ widgets/
    └─ noah/
        └─ noah_message_bubble.dart # Glassmorphism chat bubble
        └─ noah_barcode_card.dart   # 1D/QR barcode rendering widget
        └─ noah_offer_carousel.dart # Horizontal swipeable offer cards
        └─ noah_product_tile.dart   # Product comparison tile with add-to-basket
        └─ noah_route_card.dart      # Store-split route & basket savings widget

```

3. Data Models (lib/models/noah_chat_models.dart)

```
enum NoahResponseMode { text, functional, hybrid, navigation, recommendation }
```

```
class NoahEntities {
  final String? product;
  final String? merchant;
  final String? brand;
  final String? category;
  final double? priceMin;
  final double? priceMax;
  final String? location;
  final double? radius;
  final String? loyaltyCard;

```

```

  NoahEntities({
    this.product,
    this.merchant,
    this.brand,
    this.category,
    this.priceMin,
    this.priceMax,
    this.location,
    this.radius,
    this.loyaltyCard,
  });

```

```

factory NoahEntities.fromJson(Map<String, dynamic> json) {
  return NoahEntities(
    product: json['entity_product'],
    merchant: json['entity_merchant'],
    brand: json['entity_brand'],
    category: json['entity_category'],
    priceMin: json['entity_price_min']?.toDouble(),

```

```

        priceMax: json['entity_price_max']?.toDouble(),
        location: json['entity_location'],
        radius: json['entity_radius']?.toDouble(),
        loyaltyCard: json['entity_loyalty_card'],
    );
}
}

class NoahResponse {
    final String instruction;
    final String? domain;
    final String? intent;
    final String? subIntent;
    final String? tool;
    final String? responseMode;
    final bool requiresMemory;
    final bool requiresRag;
    final bool requiresRecommendation;
    final String? workflowType;
    final List<String> plannerActions;
    final int plannerActionCount;
    final List<String> toolSequence;
    final NoahEntities entities;
    final String response;

    NoahResponse({
        required this.instruction,
        this.domain,
        this.intent,
        this.subIntent,
        this.tool,
        this.responseMode,
        this.requiresMemory = false,
        this.requiresRag = false,
        this.requiresRecommendation = false,
        this.workflowType,
        this.plannerActions = const [],
        this.plannerActionCount = 0,
        this.toolSequence = const [],
        required this.entities,
        required this.response,
    });

    factory NoahResponse.fromJson(Map<String, dynamic> json) {
        return NoahResponse(
            instruction: json['instruction'] ?? '',
            domain: json['domain'],
            intent: json['intent'],
            subIntent: json['sub_intent'],
            tool: json['tool'],
            responseMode: json['response_mode'],

```

```

requiresMemory: json['requires_memory'] ?? false,
requiresRag: json['requires_rag'] ?? false,
requiresRecommendation: json['requires_recommendation'] ?? false,
workflowType: json['workflow_type'],
plannerActions: List<String>.from(json['planner_actions'] ?? []),
plannerActionCount: json['planner_action_count'] ?? 0,
toolSequence: List<String>.from(json['tool_sequence'] ?? []),
entities: NoahEntities.fromJson(json),
response: json['response'] ?? '',
);
}
}

```

```
enum MessageSender { user, noah }
```

```

class ChatMessage {
  final String id;
  final MessageSender sender;
  final String text;
  final DateTime timestamp;
  final NoahResponse? noahData;

```

```

  ChatMessage({
    required this.id,
    required this.sender,
    required this.text,
    required this.timestamp,
    this.noahData,
  });
}

```

4. API Client (lib/services/noah_api_service.dart)

```

import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/noah_chat_models.dart';

class NoahApiService {
  final String baseUrl;

  NoahApiService({this.baseUrl = 'http://127.0.0.1:8000/api'});

  Future<NoahResponse> sendInstruction(String instruction) async {
    final uri = Uri.parse('$baseUrl/chat');
    final response = await http.post(
      uri,
      headers: {'Content-Type': 'application/json'},

```

```

        body: jsonEncode({'instruction': instruction}),
    );

    if (response.statusCode == 200) {
        final data = jsonDecode(utf8.decode(response.bodyBytes));
        return NoahResponse.fromJson(data);
    } else {
        throw Exception('Noah API error: ${response.statusCode}');
    }
}

```

5. Action Dispatcher (lib/services/noah_action_dispatcher.dart)

```

import 'package:flutter/material.dart';
import '../models/noah_chat_models.dart';

class NoahActionDispatcher {
    static void dispatch(BuildContext context, NoahResponse response) {
        final actions = response.plannerActions;
        final entities = response.entities;

        for (final action in actions) {
            switch (action) {
                case 'DISPLAY_BARCODE':
                case 'OPEN_WALLET_CARD':
                    _openBarcodeModal(context, entities.loyaltyCard ?? entities.brand ?? 'PAYBACK');
                    break;
                case 'PLAN_ROUTE':
                case 'OPEN_GOOGLE_MAPS':
                    _launchNavigation(context, entities.merchant ?? entities.location ?? '');
                    break;
                case 'SEARCH_OFFERS':
                case 'OPEN_WEEKLY_FLYER':
                    // Highlight/scroll to offers section or open flyer modal
                    break;
                case 'FIND_CHEAPEST_BASKET':
                case 'OPTIMIZE_SHOPPING_ROUTE':
                    // Open Offerhopper shopping route optimizer sheet
                    break;
            }
        }
    }

    static void _openBarcodeModal(BuildContext context, String cardName) {
        // Show high-contrast barcode modal with screen brightness boost
    }
}

```

```

}

static void _launchNavigation(BuildContext context, String destination) {
  // Launch Google Maps / Apple Maps intent with destination query
}
}

```

6. State Management & Multi-Turn Context (lib/providers/noah_chat_provider.dart)

```

import 'package:flutter/foundation.dart';
import '../models/noah_chat_models.dart';
import '../services/noah_api_service.dart';

class NoahChatProvider extends ChangeNotifier {
  final NoahApiService _apiService;
  final List<ChatMessage> _messages = [];
  bool _isLoading = false;

  // Active contextual memory across conversation turns
  String? activeMerchant;
  String? activeLoyaltyCard;
  String? lastProductSearch;
  String? userLocation;

  NoahChatProvider(this._apiService);

  List<ChatMessage> get messages => List.unmodifiable(_messages);
  bool get isLoading => _isLoading;

  Future<void> sendMessage(String text) async {
    if (text.trim().isEmpty) return;

    final userMsg = ChatMessage(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      sender: MessageSender.user,
      text: text,
      timestamp: DateTime.now(),
    );
    _messages.add(userMsg);
    _isLoading = true;
    notifyListeners();

    try {
      final noahResponse = await _apiService.sendInstruction(text);

      // Update contextual memory from extracted entities

```

```

    if (noahResponse.entities.merchant != null) {
      activeMerchant = noahResponse.entities.merchant;
    }
    if (noahResponse.entities.loyaltyCard != null) {
      activeLoyaltyCard = noahResponse.entities.loyaltyCard;
    }
    if (noahResponse.entities.product != null) {
      lastProductSearch = noahResponse.entities.product;
    }

    final assistantMsg = ChatMessage(
      id: (DateTime.now().millisecondsSinceEpoch + 1).toString(),
      sender: MessageSender.noah,
      text: noahResponse.response,
      timestamp: DateTime.now(),
      noahData: noahResponse,
    );
    _messages.add(assistantMsg);
  } catch (e) {
    _messages.add(ChatMessage(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      sender: MessageSender.noah,
      text: "I'm having trouble connecting to the Noah service. Please check your connection.",
      timestamp: DateTime.now(),
    ));
  } finally {
    _isLoading = false;
    notifyListeners();
  }
}

void clearChat() {
  _messages.clear();
  activeMerchant = null;
  activeLoyaltyCard = null;
  lastProductSearch = null;
  notifyListeners();
}
}

```

7. Interactive In-Chat Widgets

A. Barcode Card (widgets/noah/noah_barcode_card.dart)

- Generates Code128 / QR Code using `barcode_widget` package.
- Displays loyalty provider logo (Payback, Lidl Plus, Rewe Bonus).
- Includes one-tap copy and full-screen brightness boost.

B. Offer Carousel (widgets/noah/noah_offer_carousel.dart)

- Horizontal `ListView` of discount tiles matching the merchant (e.g. Netto, Lidl).
- Shows original price, discounted price, expiration badge, and "Add to Basket" action.

C. Route & Basket Optimizer Card (widgets/noah/noah_route_card.dart)

- Renders Offerhopper multi-store breakdown:
 - Supermarket stops (e.g., *Stop 1: Lidl (€4.20) -> Stop 2: Aldi (€2.80)*).
 - Total basket savings vs single-store purchase.
 - "Start Navigation" button triggering Google Maps.

8. Verification & Integration Checklist

- ☐ **Barcode Display:** Saying *"Show my Payback barcode"* renders barcode widget with card number.
- ☐ **Price Filtered Search:** Saying *"Frozen pizzas under €2"* displays matching product tiles with prices.
- ☐ **Compound Multi-Step:** Saying *"Ice cream offers at Netto below €3, then show Payback card"* renders the offer carousel and presents the barcode widget.
- ☐ **Offerhopper Optimization:** Saying *"Cheapest basket for milk, eggs, bread in Berlin"* renders the multi-store savings card with `offerhopper_mcp`.
- ☐ **RAG FAQ:** Asking *"Does Payto store my payment details?"* responds accurately from the PayTo FAQ knowledge base.